

SEMESTER PROJECT REPORT

SDA-Pro

Security Incident Response & Threat Mitigation Platform

PROJECT DETAILS	
Project Title	SDA-Pro — Security Incident Response & Threat Mitigation Platform
Course	SE3213 – Software Design and Architecture
Instructor	Mr. Uzair Rasheed
Semester	Spring 2026
Technology Stack	Java Console Prototype + Enterprise Architect UML

GROUP MEMBERS	
Student Name	SAP ID
Abdul Wahab Khan	55072
Rohail Abbas	54729
Ziafat Hussain	55062

Table of Contents

1. Introduction	5
1.1 Project Background	5
1.2 Scope of the Project	5
2. Project Objectives	5
3. System Overview.....	6
3.1 End-to-End System Flow	6
3.2 Package Structure	7
4. Architecture Styles Applied.....	7
4.1 SOA Architecture	7
4.2 MVC Architecture	8
4.3 Layered Architecture	8
4.4 Event-Driven Architecture.....	8
5. Design Patterns Used	8
5.1 Creational Patterns Detail	9
5.1.1 Singleton Pattern	9
5.1.2 Factory Method Pattern	9
5.1.3 Abstract Factory Pattern	10
5.2 Structural Patterns Detail.....	10
5.2.1 Composite Pattern.....	10
5.2.2 Adapter Pattern	10
5.2.3 Proxy Pattern	10
5.2.4 Decorator Pattern.....	10
5.2.5 Facade Pattern.....	11
5.3 Behavioral Patterns Detail.....	11
5.3.1 Chain of Responsibility Pattern	11
5.3.2 State Pattern.....	11
5.3.3 Strategy Pattern.....	11
5.3.4 Observer Pattern	11
6. Implementation Package Details	12
6.1 Alert Ingestion Subsystem (src/ingestion)	12
6.2 Alert Composite Structure (src/alert/composite).....	12
6.3 Enrichment Pipeline (src/enrichment/pipeline)	12
6.4 Threat Intelligence Subsystem (src/threatintel)	13
6.5 Incident Management (src/incident)	13

6.6 Response Orchestration (src/response).....	13
6.7 Event-Driven Communication (src/events).....	14
7. Architecture Decision Records (ADRs)	14
ADR-001: SOA vs. Microservices vs. Modular Monolith	14
ADR-002: Synchronous vs. Asynchronous Communication	14
ADR-003: Database Strategy	14
ADR-004: Threat Intelligence Caching Strategy	15
ADR-005: Real-Time Push Strategy	15
8. API Contracts.....	15
8.1 Alert Ingestion API.....	15
POST /alerts/ingest.....	16
8.2 Incident Management API.....	16
POST /incidents — Create Incident	16
Lifecycle Transition Endpoints	16
8.3 Response Orchestration API.....	16
POST /incidents/{incidentId}/respond	16
8.4 Threat Intelligence API	17
GET /threat-intel/reputation/{indicator}	17
9. UML Diagrams.....	17
9.1 Class Diagram	17
9.2 Component Diagram	18
9.3 Sequence Diagram — Alert Ingestion & Enrichment.....	18
9.4 Sequence Diagram — Incident Response Orchestration	19
10. Coding Environment & Project Setup	19
10.1 Development Environment	19
10.2 IDE / VS Code Setup Screenshot	20
10.3 Compilation Steps	24
Linux / macOS / Git Bash	24
Windows PowerShell	24
10.4 Java Compilation Screenshot	24
11. Program Output & Demo Results	24
11.1 Program Output Screenshot	25
11.2 Web Dashboard Screenshot.....	26
11.3 Design Pattern Verification Table	28
12. Testing & Validation.....	28

13. Challenges & Lessons Learned	28
13.1 Challenges Encountered.....	29
13.2 Lessons Learned	29
14. Conclusion	29

1. Introduction

SDA-Pro is a simplified Security Operations Center (SOC) platform developed as a semester project for SE3213 – Software Design and Architecture at Riphah International University, Islamabad. The platform is engineered to simulate the full lifecycle of a security incident, from initial alert ingestion through enrichment, incident creation, response orchestration, and final audit logging.

Modern enterprises rely on SOC platforms to detect, analyze, and respond to cybersecurity threats in real time. SDA-Pro replicates this domain in an academically rigorous manner, deliberately applying every major architectural style and design pattern covered in the course curriculum.

The implementation uses a clean Java console prototype, intentionally kept straightforward to ensure that architectural decisions and design pattern usage remain transparent and easy to follow during code review and viva evaluation.

1.1 Project Background

Security Operations Centers are mission-critical environments where analysts monitor, detect, and respond to threats across organizational infrastructure. Key tasks performed in a SOC include ingesting alerts from diverse sources such as SIEMs, firewalls, and intrusion detection systems; normalizing those alerts into a common format; correlating and enriching them with threat intelligence data; escalating significant events into formal incidents; and executing pre-defined or adaptive response playbooks.

SDA-Pro models each of these real-world SOC tasks as a software subsystem, then demonstrates how established design patterns and architectural styles can be applied to solve the challenges that arise in each subsystem. The result is a working prototype that is simultaneously a functional security tool simulation and a comprehensive showcase of software architecture principles.

1.2 Scope of the Project

The scope of SDA-Pro is bounded by the following capabilities:

- Ingestion of security alerts from at least two distinct mock sources (Splunk SIEM and Palo Alto Firewall).
- Normalization of heterogeneous alert formats into a unified canonical model.
- Enrichment of alerts through a multi-stage pipeline including deduplication, geo-IP lookup, threat intelligence, asset context resolution, and severity classification.
- Grouping of related alerts into composite structures such as campaigns and clusters.
- Creation and lifecycle management of formal security incidents.
- Orchestration of response actions using strategy selection, factory creation, decorator enhancement, and proxy-enforced authorization.
- Real-time event-driven notifications to dashboard, audit, notification, and metrics subsystems.
- Documentation of all architectural decisions through Architecture Decision Records (ADRs).

2. Project Objectives

The primary objectives of SDA-Pro are drawn from the course requirements and expanded to reflect best practices in enterprise software design. The system is designed to satisfy the following objectives:

1. Ingest alerts from at least two mock security sources, simulating a real SOC environment with multiple data feeds.

2. Normalize different alert formats into a single canonical alert model using the Adapter and Factory Method patterns.
3. Enrich alerts through a Chain of Responsibility pipeline that applies deduplication, geo-IP, threat intelligence, asset context, and classification handlers independently.
4. Group related alerts using the Composite pattern, supporting both individual alerts and grouped campaigns or clusters.
5. Create incidents from alert groups and control their full lifecycle (New, Under Triage, Containment, Eradication, Recovery, Closed) using the State pattern.
6. Execute response actions using the Strategy pattern for plan selection, Factory Method for action creation, Decorator for capability enhancement, and Proxy for authorization control.
7. Update dashboard and audit modules through a loosely coupled Observer/EventBus mechanism, ensuring separation of concerns.
8. Document and justify all major architectural decisions through five ADR files covering SOA boundaries, communication style, database strategy, threat intel caching, and real-time push strategy.
9. Provide comprehensive API contracts for all major service interfaces.
10. Deliver a working Java prototype with console demonstration output confirming end-to-end flow.

Technology Stack

Language: Java (JDK 11+)

Architecture Styles: SOA, MVC, Layered, Event-Driven

Design Patterns: 12 patterns applied

UML Tooling: Enterprise Architect / PlantUML

Demo Entry Point: MainDemo.java | Web Dashboard: WebDashboardServer.java

3. System Overview

SDA-Pro is structured as a modular SOA-style prototype implemented within a single Java codebase. Each major functional area is encapsulated in a dedicated package and exposed through a well-defined service class. The system simulates the full end-to-end operation of a Security Operations Center from raw alert ingestion through to audit-logged response execution.

3.1 End-to-End System Flow

The system processes security threats through the following sequential pipeline:

```
Mock Splunk / Firewall Alert
→ Adapter      : Converts external format into RawAlert
→ Factory Method : Selects correct AlertNormalizer
→ CanonicalAlert : Unified internal alert model
→ Composite     : Groups alerts into AlertCampaign / IncidentCluster
→ Chain of Resp. : Dedup → GeoIP → ThreatIntel → AssetCtx → Classify
→ Proxy + Adapter : CachingThreatIntelProxy → VirusTotal/MISP
→ Incident      : Created and placed in NewState
→ State Machine  : New → Triage → Containment → Eradication → Closed
→ Strategy       : ResponseStrategy selected based on severity
→ Factory        : ResponseActionFactory creates actions
→ Decorator      : Audit + Approval + Rollback + Metrics wrapping
→ Proxy          : AuthorizedResponseActionProxy checks authorization
→ Facade         : IncidentResponseFacade orchestrates the above
→ EventBus       : Publishes domain events
→ Observers      : Dashboard / Audit / Notification / Metrics updated
```

3.2 Package Structure

The codebase is organized into the following top-level packages, each corresponding to a major SOC subsystem or architectural concern:

Package	Responsibility
src/ingestion	Alert adapters, normalizers, ingestion service, configuration singleton
src/alert/composite	Composite alert structure for campaigns and clusters
src/enrichment/pipeline	Chain of Responsibility enrichment handlers
src/threatintel	Threat intel adapters, proxy, and cache
src/incident	Incident domain model and lifecycle state machine
src/response	Response strategies, actions, factory, decorators, proxy, facade
src/events	Observer interfaces, event bus, and event subscribers
src/dashboard	MVC controller, model, and view classes
src/audit	Audit service and repository for log records
src/notification	Abstract and enterprise notification factories
src/shared	Shared domain models and utility classes

4. Architecture Styles Applied

SDA-Pro deliberately demonstrates four distinct architectural styles, each applied in a different subsystem to illustrate how real enterprise platforms combine multiple approaches.

Architecture Style	Application in SDA-Pro
Service-Oriented Architecture (SOA)	Major capabilities separated into distinct service classes: AlertIngestionService, EnrichmentCorrelationService, IncidentManagementService, ResponseOrchestrationService, NotificationService, and AuditService. Each service has a defined boundary and API contract.
Model-View-Controller (MVC)	The dashboard package is divided into controllers (DashboardController), models (DashboardModel), and views (DashboardView), providing a clean separation between data, presentation, and interaction logic.
Layered Architecture	Code is vertically separated into adapter layer, service layer, domain layer, repository/cache layer, and presentation layer. Dependencies flow strictly downward.
Event-Driven Architecture	EventBusPublisher publishes domain events (ALERT_INGESTED, INCIDENT_CREATED, RESPONSE_ACTION_EXECUTED) to registered observers including DashboardUpdater, AuditLogger, NotificationDispatcher, and MetricsCollector.

4.1 SOA Architecture

The Service-Oriented Architecture style is the primary organizing principle of SDA-Pro. Each major SOC capability is encapsulated in an independent service class with its own responsibility domain. This design mirrors how real enterprise SOC platforms are built, where each service can theoretically be deployed, versioned, and scaled independently.

- **AlertIngestionService:** Owns all alert ingestion logic including adapter invocation, normalization, and composite grouping.
- **EnrichmentCorrelationService:** Orchestrates the enrichment pipeline and manages correlation of related alerts.
- **IncidentManagementService:** Creates incidents from alert groups and manages state transitions throughout the lifecycle.
- **ResponseOrchestrationService:** Coordinates strategy selection, action factory, decorator application, and proxy enforcement.
- **ThreatIntelService:** Provides a unified interface for threat intelligence queries, backed by the caching proxy.
- **NotificationService:** Abstracts all notification channel creation and dispatch through the Abstract Factory pattern.
- **AuditService:** Receives domain events and persists structured audit records.

4.2 MVC Architecture

The dashboard subsystem in SDA-Pro implements a textbook Model-View-Controller pattern. The **DashboardModel** holds the state of the dashboard including active alert counts, incident statuses, and response action outcomes. The **DashboardController** processes incoming update events from the **EventBus** and applies changes to the model. The **DashboardView** formats and renders the current model state to the console output, simulating a SOC analyst's dashboard screen.

4.3 Layered Architecture

The layered architecture is enforced through the package structure itself. At the top sits the adapter layer, which handles integration with external data sources. Below that is the service layer, which implements core business logic. The domain layer holds canonical models such as **CanonicalAlert** and **Incident**. At the bottom sits the repository and cache layer, which abstracts persistence and caching concerns. This layering ensures that each layer only depends on the layer directly below it, making the system maintainable and testable.

4.4 Event-Driven Architecture

Event-driven communication is used for all side-effect notifications in SDA-Pro. When the core business flow reaches a significant milestone such as alert ingestion completion, incident creation, or response action execution, it publishes a domain event to the **EventBusPublisher**. Registered subscribers independently handle each event without the publisher needing to know their identities or implementations. In the prototype, the event bus is in-memory. The architecture is designed to be transparently replaceable with a message broker such as **RabbitMQ** or **Apache Kafka** for a production-scale deployment.

5. Design Patterns Used

SDA-Pro applies twelve distinct design patterns from the GoF (Gang of Four) catalog and related architectural pattern literature. Each pattern is applied to a specific problem in the security incident response domain, ensuring that pattern usage is purposeful and not superficial.

Pattern	Key Classes	Purpose in SDA-Pro
Singleton	IngestionConfigManager, ThreatIntelCache, EventBusPublisher	Ensures one shared instance for global configuration, threat intel cache, and event bus across all services.
Factory Method	AlertNormalizerFactory, ResponseActionFactory	Selects and creates the correct normalizer or response action based on runtime type, without coupling callers to concrete implementations.
Abstract Factory	NotificationFactory, EnterpriseNotificationFactory	Creates families of related notification channel objects (email, SMS, SIEM connector) without specifying concrete classes.
Composite	AlertComponent, SingleAlert, AlertCampaign, IncidentCluster	Treats individual alerts and grouped alert campaigns uniformly through a common interface, enabling recursive grouping.
Adapter	SplunkAdapter, FirewallAdapter, VirusTotalAdapter, MISPAdapter	Converts incompatible external system formats into the canonical internal model expected by SDA-Pro.
Proxy	CachingThreatIntelProxy, AuthorizedResponseActionProxy	Adds caching before threat intel calls and enforces authorization checks before response actions execute.
Chain of Responsibility	EnrichmentHandler, DeduplicationHandler, GeolIPHandler, ThreatIntelHandler, AssetContextHandler, ClassificationHandler	Processes each alert through an independent sequence of enrichment steps, where each handler decides whether to act and whether to pass to the next handler.
State	IncidentState, NewState, UnderTriageState, ContainmentState, EradicationState, RecoveryState, ClosedState	Controls the valid lifecycle transitions of an Incident, preventing illegal state jumps and encapsulating transition logic within each state class.
Strategy	ResponseStrategy, AggressiveContainmentStrategy, ConservativeContainmentStrategy	Selects the correct response plan at runtime based on incident severity and organizational policy, without conditional branching in the service layer.
Decorator	AuditLogDecorator, ApprovalGateDecorator, RollbackDecorator, MetricsDecorator	Dynamically wraps response actions with additional capabilities including pre/post audit logging, approval gates, rollback snapshots, and performance metrics.
Facade	IncidentResponseFacade	Provides a single, simplified entry point for the full response orchestration flow, hiding the complexity of strategy selection, factory creation, decorator stacking, and proxy enforcement.
Observer	EventBusPublisher, DashboardUpdater, AuditLogger, NotificationDispatcher, MetricsCollector	Decouples event producers from consumers; when a domain event is published, all registered subscribers are notified without tight coupling.

5.1 Creational Patterns Detail

5.1.1 Singleton Pattern

The Singleton pattern is applied in three distinct locations within SDA-Pro. IngestionConfigManager holds the global alert ingestion configuration that all ingestion services share. ThreatIntelCache is a global cache keyed by IP address or domain, preventing repeated external queries for the same indicator. EventBusPublisher is the single event bus instance to which all publishers write and from which all subscribers read. Each of these singletons uses private constructors and static getInstance() methods, ensuring that the JVM never holds more than one instance of each.

5.1.2 Factory Method Pattern

AlertNormalizerFactory applies the Factory Method pattern to decouple the ingestion service from concrete normalizer implementations. When a raw alert arrives with a sourceType field, the factory examines the type and instantiates the appropriate normalizer: SplunkNormalizer for SIEM alerts or FirewallNormalizer for network device alerts. This design means that adding a new alert source requires only a new normalizer class and a new case in the factory, with no changes to the ingestion service itself.

ResponseActionFactory follows the same pattern for response actions, creating BlockIPAction, IsolateEndpointAction, ForcePasswordResetAction, or QuarantineAction based on the selected response strategy.

5.1.3 Abstract Factory Pattern

NotificationFactory and its concrete subclass EnterpriseNotificationFactory demonstrate the Abstract Factory pattern. The abstract factory declares creation methods for all notification channel types: createEmailNotifier(), createSmsNotifier(), and createSiemConnector(). The enterprise implementation creates production-grade variants of each channel. A mock implementation could create test stubs. The NotificationService depends only on the abstract factory interface, making it straightforward to swap notification infrastructure without touching service logic.

5.2 Structural Patterns Detail

5.2.1 Composite Pattern

The Composite pattern addresses the challenge of treating a single alert and a group of related alerts uniformly. AlertComponent defines the common interface with methods such as getSeverity(), getAlertIds(), and display(). SingleAlert is the leaf implementation, wrapping one CanonicalAlert. AlertCampaign is the composite implementation, holding a list of AlertComponent children. This design allows the enrichment pipeline and incident creation logic to operate on any AlertComponent without needing to know whether it represents one alert or a campaign of fifty related alerts.

5.2.2 Adapter Pattern

Four adapter classes bridge the gap between external data source formats and SDA-Pro's internal canonical model. SplunkAdapter retrieves a mock Splunk SIEM alert in the format that Splunk would produce and transforms it into a RawAlert understood by SDA-Pro. FirewallAdapter performs the same service for Palo Alto Firewall format alerts. VirusTotalAdapter wraps the VirusTotal API response format into SDA-Pro's ReputationResult model. MISPAdapter wraps MISP threat intelligence responses. Each adapter implements the same SourceAdapter interface, so the ingestion and threat intel services are never aware of which external system they are communicating with.

5.2.3 Proxy Pattern

CachingThreatIntelProxy sits between the enrichment pipeline and the real threat intelligence adapters. Before forwarding a query to VirusTotalAdapter or MISPAdapter, the proxy checks ThreatIntelCache for a stored result. A cache hit returns the stored result immediately, a cache miss delegates to the real adapter and stores the result for future requests. AuthorizedResponseActionProxy wraps every response action execution with an authorization check that validates the role of the requesting analyst before allowing the action to proceed. This ensures that no response action, regardless of how it was created or decorated, can execute without passing the authorization gate.

5.2.4 Decorator Pattern

The Decorator pattern is applied to response actions to add cross-cutting capabilities without modifying the core action classes. The base ResponseAction interface defines execute(). Concrete actions such as BlockIPAction implement this interface directly. When an action is prepared for execution, it is progressively wrapped by decorators in the following order from innermost to outermost: RollbackDecorator saves a system snapshot before execution so the action can be reversed if needed. AuditLogDecorator records a pre-action audit entry,

delegates to the inner action, then records a post-action audit entry. ApprovalGateDecorator checks whether an analyst approval has been granted before allowing execution to proceed. MetricsDecorator records the wall-clock time of the action execution. This stacking approach means each decorator can be applied or removed independently.

5.2.5 Facade Pattern

IncidentResponseFacade provides a single orchestration method that coordinates the entire response flow for a given incident. Internally, the facade calls the ResponseStrategy to select a plan, the ResponseActionFactory to create the required actions, stacks the appropriate decorators, passes each action through the AuthorizedResponseActionProxy, executes the actions, and publishes the RESPONSE_ACTION_EXECUTED event. Without the facade, the calling code in IncidentManagementService would need to know about strategies, factories, decorators, and proxies. The facade eliminates this coupling entirely.

5.3 Behavioral Patterns Detail

5.3.1 Chain of Responsibility Pattern

The enrichment pipeline is implemented as a Chain of Responsibility. The abstract EnrichmentHandler base class holds a reference to the next handler and provides a passToNext() method. Concrete handlers include DeduplicationHandler (checks if an identical alert has already been processed), GeoIPHandler (maps source IP to geographic region and flags high-risk geolocations), ThreatIntelHandler (invokes the CachingThreatIntelProxy to retrieve an IP reputation score), AssetContextHandler (resolves the destination IP to an internal asset record and determines asset criticality), and ClassificationHandler (upgrades the alert severity based on the combined enrichment results). Each handler processes what it is responsible for and then passes control to the next handler in the chain.

5.3.2 State Pattern

The Incident class delegates all lifecycle behavior to its current IncidentState implementation. Six state classes implement the IncidentState interface: NewState represents a freshly created incident awaiting triage. UnderTriageState represents an incident being analyzed by an analyst. ContainmentState represents an incident where containment actions are being executed. EradicationState represents an incident where the threat is being removed from the environment. RecoveryState represents an incident where affected systems are being restored. ClosedState is the terminal state where all response and recovery activities are complete. Each state class implements the methods that are valid in that state and throws an exception or logs a warning when an illegal transition is attempted.

5.3.3 Strategy Pattern

ResponseOrchestrationService applies the Strategy pattern to response plan selection. The ResponseStrategy interface declares a selectActions() method. AggressiveContainmentStrategy is selected for CRITICAL and HIGH severity incidents; it returns a comprehensive set of actions including IP blocking, endpoint isolation, and credential reset. ConservativeContainmentStrategy is selected for MEDIUM incidents; it returns a more measured set of actions to minimize operational disruption. The service depends only on the ResponseStrategy interface, so new strategies can be added without modifying existing code.

5.3.4 Observer Pattern

EventBusPublisher implements a classic Observer pattern. It maintains a registry mapping event types to lists of EventSubscriber implementations. When a domain event is published, the bus iterates over all registered subscribers for that event type and invokes their onEvent() method. DashboardUpdater updates the dashboard model, AuditLogger persists an audit record, NotificationDispatcher sends the appropriate notification, and MetricsCollector increments performance counters. Adding a new subscriber requires only implementing the EventSubscriber interface and registering it with the bus; no changes are needed to any publisher.

6. Implementation Package Details

This section provides a detailed walkthrough of each implementation package, describing the classes within, their responsibilities, and how they collaborate to fulfill their subsystem's function.

6.1 Alert Ingestion Subsystem (src/ingestion)

The ingestion subsystem is the entry point of the entire SDA-Pro pipeline. Its responsibility is to receive raw security signals from external sources, convert them to a canonical format, and emit them for downstream processing.

- **AlertIngestionService:** The top-level service orchestrating ingestion. It calls source adapters to fetch raw alerts, invokes **AlertNormalizerFactory** to obtain the appropriate normalizer, and publishes the **ALERT_INGESTED** event once normalization is complete.
- **SplunkAdapter:** Implements the **SourceAdapter** interface for Splunk SIEM. Simulates fetching a raw JSON-like Splunk alert and returns a **RawAlert** object.
- **FirewallAdapter:** Implements the **SourceAdapter** interface for Palo Alto Firewall. Simulates fetching a raw firewall log entry and returns a **RawAlert** object.
- **AlertNormalizerFactory:** Examines the **sourceType** field of a **RawAlert** and instantiates the correct **AlertNormalizer** implementation.
- **SplunkNormalizer / FirewallNormalizer:** Convert source-specific **RawAlert** fields into a standardized **CanonicalAlert** with consistent fields including **alertId**, **severity**, **sourceIp**, **destinationIp**, **timestamp**, and **sourceType**.
- **IngestionConfigManager:** Singleton holding ingestion configuration such as source addresses, retry limits, and polling intervals.
- **CanonicalAlert:** The shared domain model produced by all normalizers and consumed by all downstream subsystems.

6.2 Alert Composite Structure (src/alert/composite)

The composite subsystem allows SDA-Pro to group related alerts from multiple sources into a single logical unit for enrichment and incident creation purposes.

- **AlertComponent:** The abstract base interface with **getSeverity()**, **getAlertIds()**, and **display()** methods, implemented by both leaf and composite nodes.
- **SingleAlert:** The leaf implementation wrapping one **CanonicalAlert**.
- **AlertCampaign:** The composite implementation that holds a list of **AlertComponent** children and aggregates severity from across all children.
- **IncidentCluster:** An alternate composite grouping alerts by incident theme rather than campaign origin.

6.3 Enrichment Pipeline (src/enrichment/pipeline)

The enrichment pipeline applies a Chain of Responsibility to progressively add context to a **CanonicalAlert** before incident creation.

- **EnrichmentHandler:** Abstract base class with a reference to the next handler and a **passToNext()** delegation method.
- **DeduplicationHandler:** Checks a deduplication cache keyed by alert signature to prevent the same alert from being processed twice.
- **GeoIPHandler:** Maps the source IP address to a geographic region and flags addresses from high-risk regions.

- ThreatIntelHandler: Invokes the CachingThreatIntelProxy to retrieve a threat reputation score for the source IP.
- AssetContextHandler: Resolves the destination IP to an internal asset record and attaches asset criticality metadata to the alert.
- ClassificationHandler: Synthesizes all enrichment metadata to produce a final severity classification, potentially upgrading from HIGH to CRITICAL based on compound risk factors.

6.4 Threat Intelligence Subsystem (src/threatintel)

The threat intelligence subsystem provides reputation and context information for IP addresses and domains observed in security alerts.

- ThreatIntelService: The service interface for all threat intelligence queries.
- VirusTotalAdapter: Adapts the VirusTotal API response format to SDA-Pro's ReputationResult domain model.
- MISPAdapter: Adapts the MISP open-source threat intelligence platform response to ReputationResult.
- CachingThreatIntelProxy: Proxy that checks ThreatIntelCache before delegating to a real adapter. On cache miss, stores the result for future queries.
- ThreatIntelCache: Singleton in-memory cache keyed by indicator value (IP or domain), holding ReputationResult objects with a configurable TTL.

6.5 Incident Management (src/incident)

The incident management subsystem owns the creation and lifecycle control of formal security incidents.

- Incident: Core domain entity with an incidentId, associated AlertComponent, severity, current state, and assigned analyst.
- IncidentState: Interface declaring lifecycle methods: beginTriage(), startContainment(), startEradication(), startRecovery(), close().
- NewState, UnderTriageState, ContainmentState, EradicationState, RecoveryState, ClosedState: Concrete state implementations.
- IncidentManagementService: Creates Incident objects from AlertCampaign composites, drives state transitions, and coordinates with ResponseOrchestrationService and AuditService.

6.6 Response Orchestration (src/response)

The response orchestration subsystem selects, creates, enhances, and executes the response actions appropriate for each incident.

- ResponseStrategy, AggressiveContainmentStrategy, ConservativeContainmentStrategy: Strategy pattern implementations.
- ResponseAction: The base interface for all executable response actions.
- BlockIPAction, IsolateEndpointAction, ForcePasswordResetAction, QuarantineAction: Concrete response action implementations.
- ResponseActionFactory: Factory Method for creating action instances by type string.
- AuditLogDecorator, ApprovalGateDecorator, RollbackDecorator, MetricsDecorator: Decorator implementations wrapping ResponseAction.
- AuthorizedResponseActionProxy: Proxy verifying analyst role before execution.
- IncidentResponseFacade: Facade coordinating strategy, factory, decorator stacking, proxy, and event publishing.

6.7 Event-Driven Communication (src/events)

- DomainEvent: Value object carrying eventType, payload string, and timestamp.
- EventSubscriber: Interface with onEvent(DomainEvent) method.
- EventBusPublisher: Singleton maintaining subscriber registry and publish/subscribe methods.
- DashboardUpdater: Subscriber that calls DashboardController with event data.
- AuditLogger: Subscriber that persists audit records to AuditRepository.
- NotificationDispatcher: Subscriber that dispatches external notifications via NotificationFactory.
- MetricsCollector: Subscriber that increments performance and operational metrics counters.

7. Architecture Decision Records (ADRs)

Architecture Decision Records document the significant architectural choices made during SDA-Pro's design. Each ADR follows a standard structure of Status, Context, Decision, Rationale, and Consequences. Five ADRs are maintained in the docs/adr directory.

ADR-001: SOA vs. Microservices vs. Modular Monolith

Status	Accepted
Context	SDA-Pro contains several major capabilities: alert ingestion, enrichment, incident management, threat intelligence, response orchestration, notifications, audit logging, and dashboard updates. The course requires demonstration of SOA boundaries, MVC, layered architecture, event-driven communication, and all required design patterns.
Decision	A modular SOA-style prototype implemented in one Java codebase. Each major capability is represented as a separate service package.
Rationale	Gives clear service boundaries without over-engineering. Allows SOA demonstration through component diagrams, package structure, service responsibilities, and API contracts, while keeping the code simple enough to explain in viva.
Consequences	Easier to run and demonstrate. Service boundaries visible in code and diagrams. A pure microservices deployment is avoided due to semester scope. Communication is simulated through direct service calls and an in-memory event bus.

ADR-002: Synchronous vs. Asynchronous Communication

Status	Accepted
Context	Some operations require an immediate result (normalizing alerts, selecting a strategy). Others should notify multiple modules without tight coupling (dashboard updates, audit logging, notifications, metrics).
Decision	Hybrid approach: synchronous calls for direct request/response operations; asynchronous event-style communication for domain events through EventBusPublisher.
Rationale	Synchronous calls keep the main business flow simple. Event-driven communication is better for side effects where the publisher should not know all receivers.
Consequences	Core flow remains debuggable. Event subscribers are loosely coupled. In production, the in-memory event bus can be replaced with RabbitMQ or Apache Kafka.

ADR-003: Database Strategy

Status	Accepted
--------	----------

Context	A real SOC platform needs persistent storage for incidents, alerts, audit logs, user sessions, and threat intelligence cache. Production recommendations include PostgreSQL for relational records and Redis for caching.
Decision	In-memory repositories and in-memory cache for the prototype. Architecture documents the intended production database choices: PostgreSQL for incident and audit records, Redis for threat intelligence cache and sessions.
Rationale	In-memory storage keeps the prototype runnable on any student machine without external dependencies. Repository and service boundaries are separated so production replacements are straightforward.
Consequences	Demo runs without external database setup. Data is not persisted after program termination. Architecture remains extendable with real database backends.

ADR-004: Threat Intelligence Caching Strategy

Status	Accepted
Context	Threat intelligence providers such as VirusTotal and MISP may be slow, rate-limited, or expensive to query. The same IP address or domain may appear in multiple alerts, so repeated lookups should be avoided.
Decision	Proxy-based caching strategy through CachingThreatIntelProxy. The proxy checks ThreatIntelCache before calling the real provider adapter.
Rationale	Naturally applies the Proxy Pattern. The client does not call VirusTotal or MISP directly; the proxy controls access and delegates only when required.
Consequences	Repeated IP reputation checks are faster. Unnecessary external provider calls are avoided. Cache can be replaced with Redis in production. Proxy can be extended with rate limiting.

ADR-005: Real-Time Push Strategy

Status	Accepted
Context	SOC analysts need fast updates when alerts arrive, incidents are created, or response actions execute. Options include polling, Server-Sent Events, WebSockets, and message brokers.
Decision	Observer-based in-memory event bus for the prototype. Production mapping to WebSockets/SSE for dashboard push and RabbitMQ/Kafka for service-level event distribution.
Rationale	Observer is simple, demonstrable, and directly required by the course. Proves loose coupling without external infrastructure for the prototype.
Consequences	Dashboard updates are visible in console output. No external message broker required for the demo. Real deployment can replace the in-memory bus with production-grade infrastructure.

8. API Contracts

SDA-Pro defines formal API contracts for all major service interfaces. These contracts describe the expected request and response formats for each operation, serving as the interface specification between subsystems and as the basis for future external integrations.

8.1 Alert Ingestion API

POST /alerts/ingest

Accepts a raw security alert from an external source and returns a normalized canonical alert record.

```
Request:
{
  "sourceType":    "SPLUNK",
  "rawMessage":    "Failed login detected from suspicious IP",
  "sourceIp":      "185.21.10.5",
  "destinationIp": "10.0.0.15",
  "username":      "admin",
  "timestamp":     "2026-05-29T10:30:00Z"
}

Response:
{
  "alertId":       "ALERT-1001",
  "sourceType":    "SPLUNK",
  "severity":      "HIGH",
  "status":        "NORMALIZED",
  "message":       "Alert ingested and normalized successfully"
}
```

8.2 Incident Management API

POST /incidents — Create Incident

```
Request:
{
  "alertGroupId":  "CAMPAIGN-APT-001",
  "title":         "Suspicious login campaign",
  "severity":      "CRITICAL"
}

Response:
{
  "incidentId":    "INC-1001",
  "state":         "NEW",
  "severity":      "CRITICAL",
  "message":       "Incident created in NewState"
}
```

Lifecycle Transition Endpoints

Endpoint	State Transition
POST /incidents/{id}/triage	NEW → UNDER_TRIAGE
POST /incidents/{id}/containment	UNDER_TRIAGE → CONTAINMENT
POST /incidents/{id}/eradication	CONTAINMENT → ERADICATION
POST /incidents/{id}/recovery	ERADICATION → RECOVERY
POST /incidents/{id}/close	RECOVERY → CLOSED

8.3 Response Orchestration API

POST /incidents/{incidentId}/respond

```
Request:
{
  "analystRole":   "SOC_MANAGER",
  "strategyHint":  "AGGRESSIVE"
}
```



```
}

Response:
{
  "incidentId": "INC-1001",
  "actionsExecuted": ["BlockIPAction", "IsolateEndpointAction"],
  "outcomes": ["SUCCESS", "SUCCESS"],
  "auditRef": "AUDIT-20260529-001"
}
```

8.4 Threat Intelligence API

GET /threat-intel/reputation/{indicator}

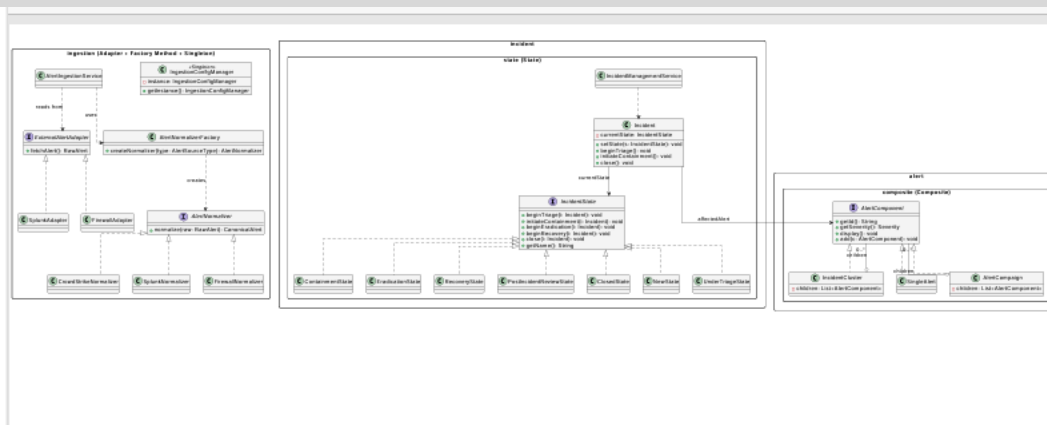
```
Response:
{
  "indicator": "185.21.10.5",
  "score": 90,
  "verdict": "MALICIOUS",
  "source": "VirusTotal",
  "cachedAt": "2026-05-29T10:28:00Z"
}
```

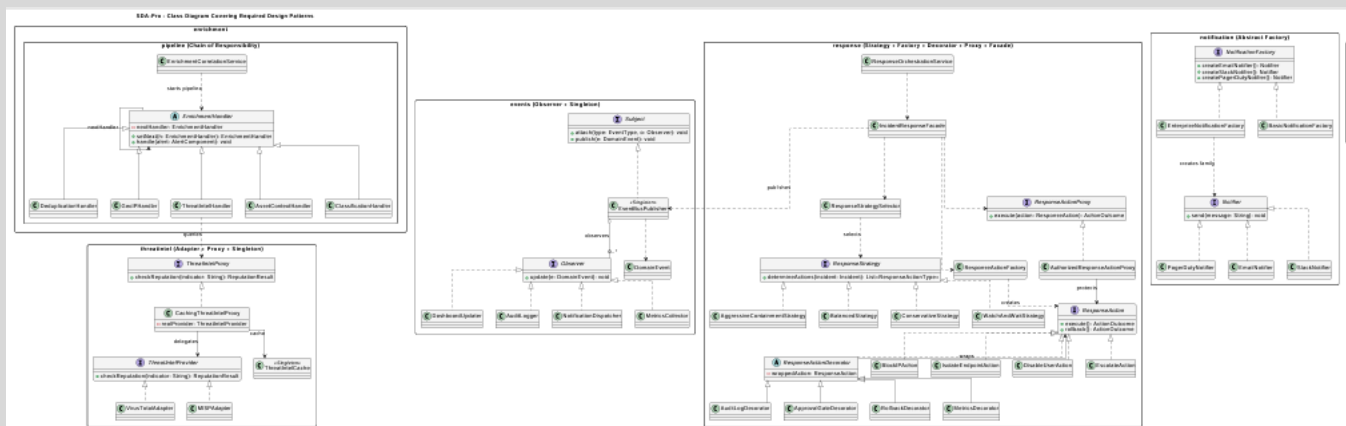
9. UML Diagrams

SDA-Pro is accompanied by four primary UML diagrams produced using Enterprise Architect and PlantUML source files located in the docs/uml directory. These diagrams capture the structural and behavioral design of the system at different levels of abstraction.

9.1 Class Diagram

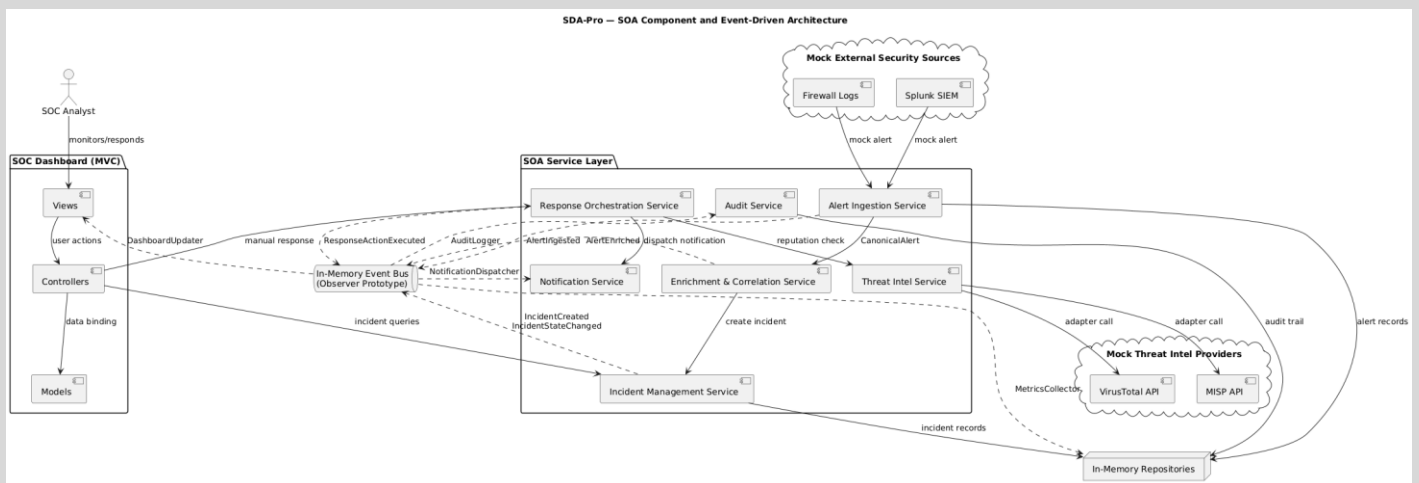
The class diagram shows all major classes and interfaces in SDA-Pro, organized by package. It illustrates the inheritance hierarchies for design patterns including Adapter, State, Strategy, Decorator, Composite, and Observer, as well as the associations and dependencies between service classes, domain models, and utility classes.





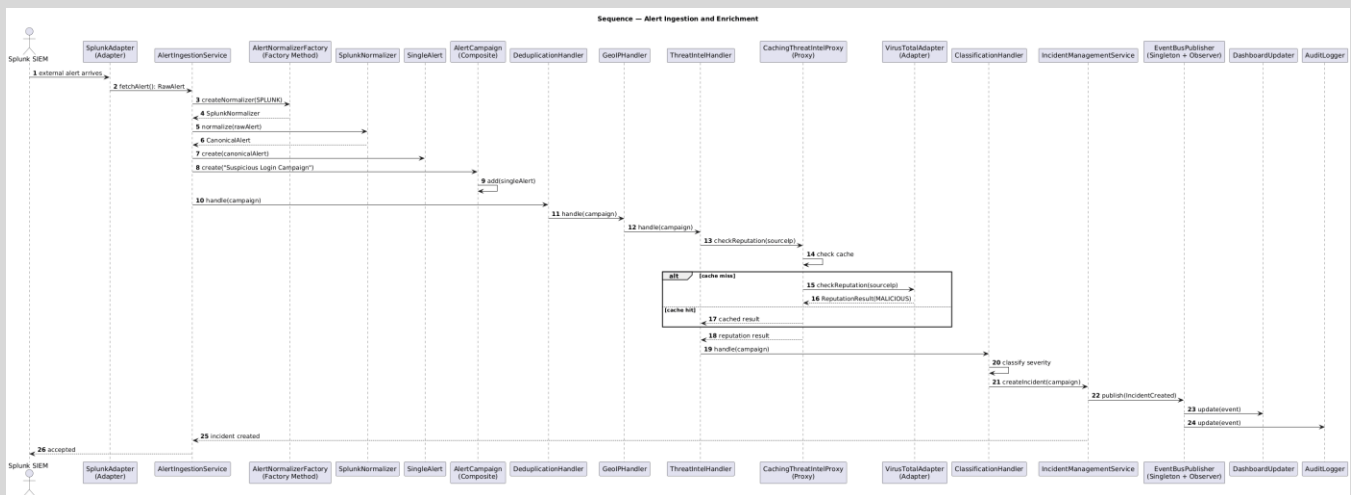
9.2 Component Diagram

The component diagram presents the SOA view of SDA-Pro, showing each service component, the interfaces it provides and requires, and the inter-component communication paths. The diagram includes the EventBus as a central communication component and shows the separation between the ingestion, enrichment, incident, response, notification, and audit components.



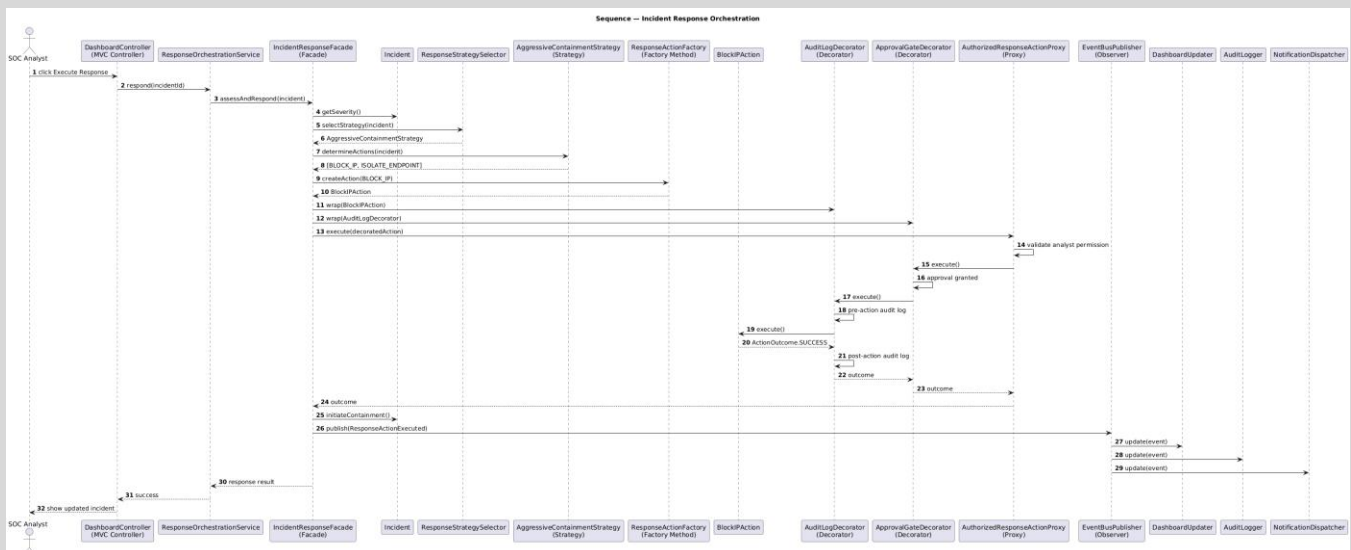
9.3 Sequence Diagram — Alert Ingestion & Enrichment

This sequence diagram traces the complete message flow from the moment a raw alert is received by SplunkAdapter through normalization by AlertNormalizerFactory, composite grouping by AlertCampaign, and sequential enrichment by each handler in the Chain of Responsibility pipeline. It shows the cache-miss path through CachingThreatIntelProxy to VirusTotalAdapter and the resulting ALERT_ENRICHED event publication.



9.4 Sequence Diagram — Incident Response Orchestration

This sequence diagram traces the response orchestration flow initiated by IncidentResponseFacade. It shows the strategy selection, action creation by ResponseActionFactory, progressive decorator wrapping, authorization check by AuthorizedResponseActionProxy, execution of the decorated action, and the cascade of event notifications to all subscribers through EventBusPublisher.



10. Coding Environment & Project Setup

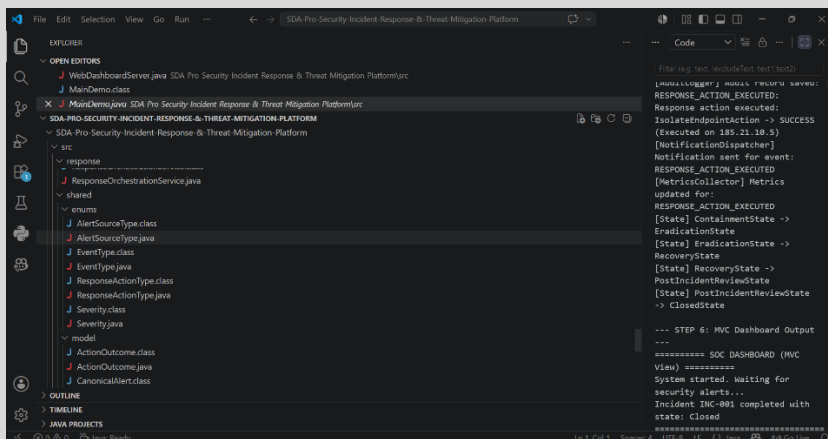
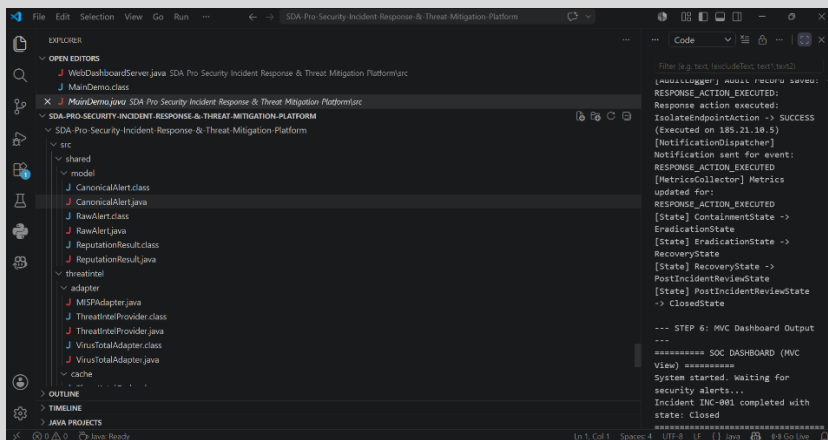
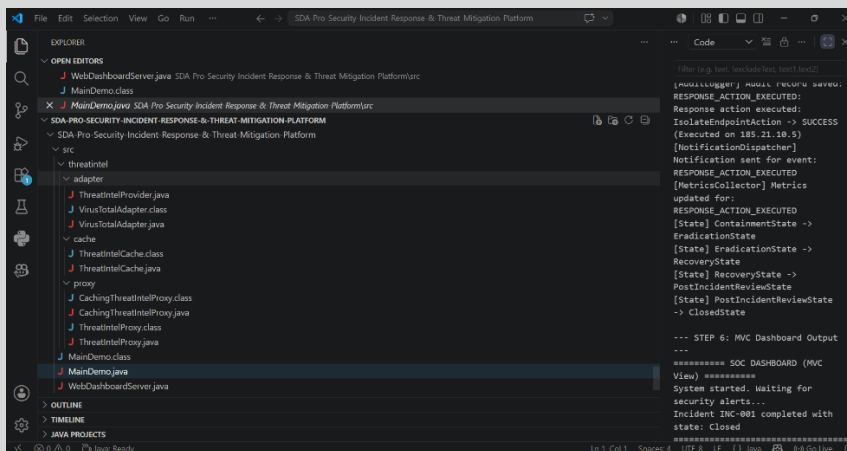
SDA-Pro is developed using standard Java tooling and requires no external framework dependencies beyond the Java Development Kit. The project structure is organized to be immediately runnable on any developer machine with JDK 11 or above.

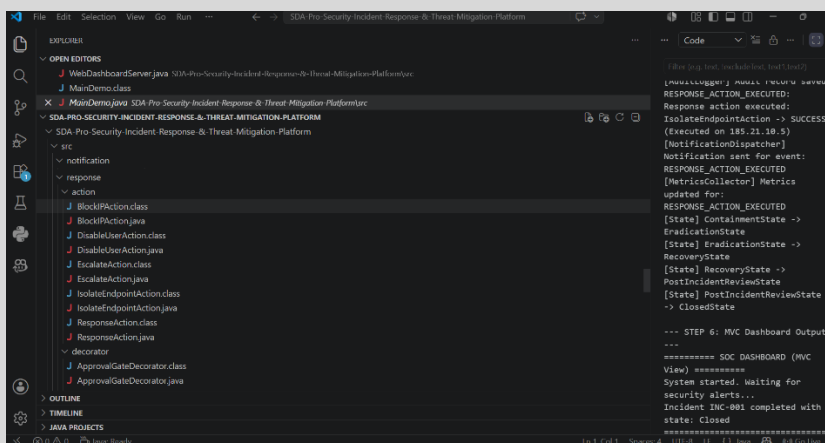
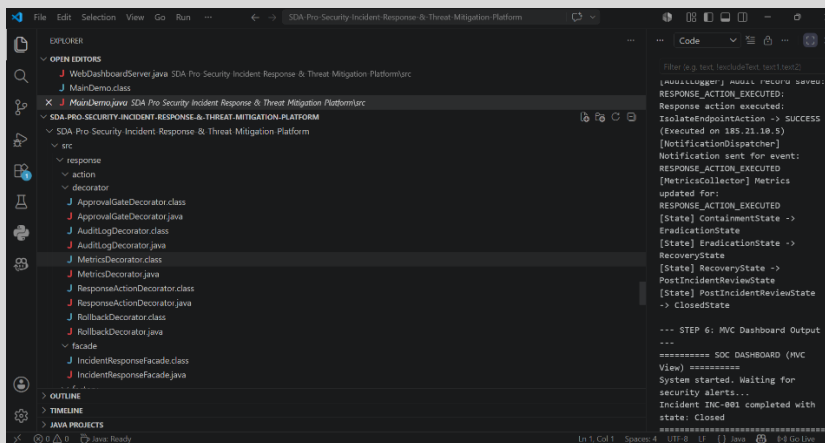
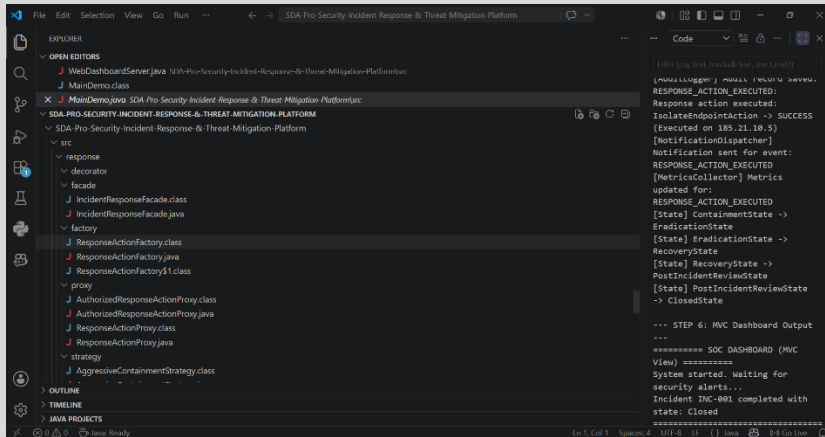
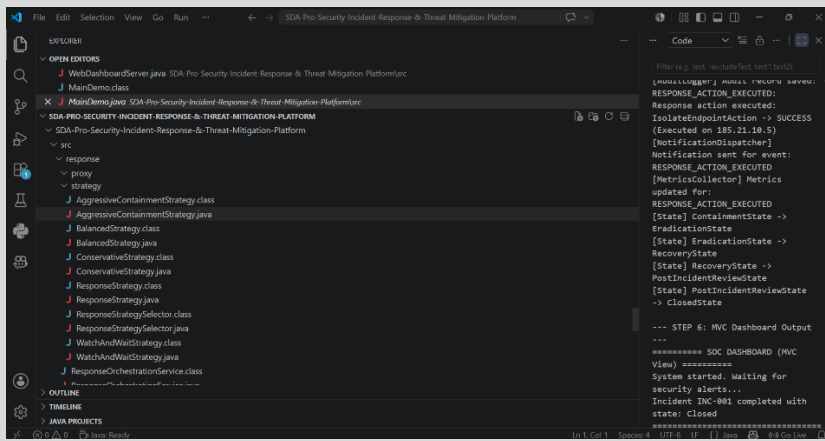
10.1 Development Environment

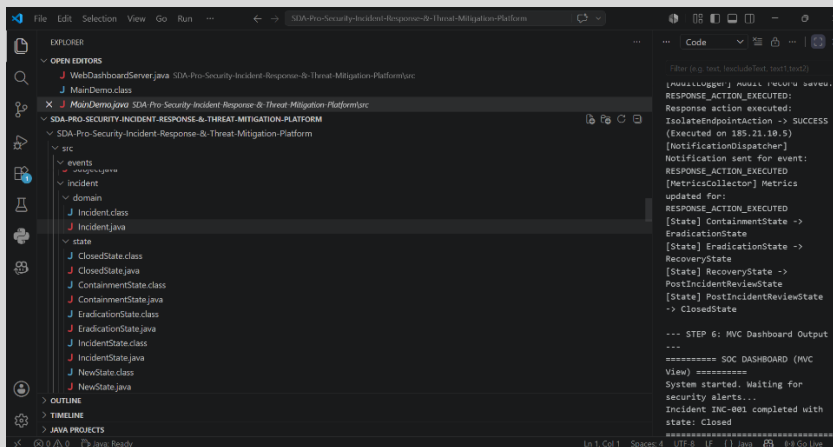
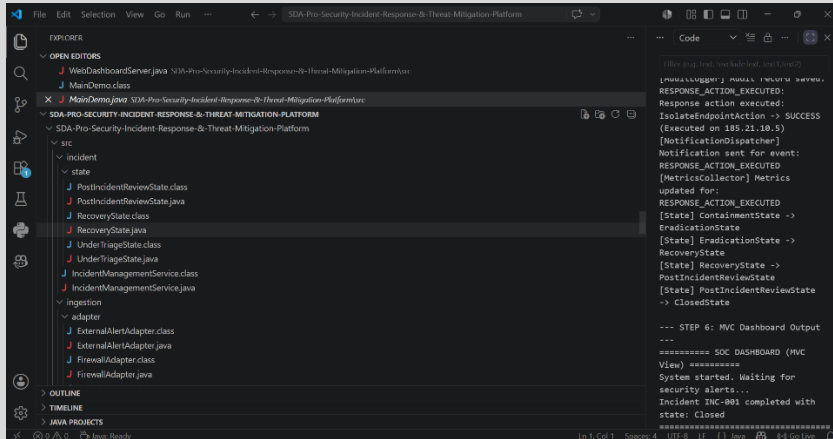
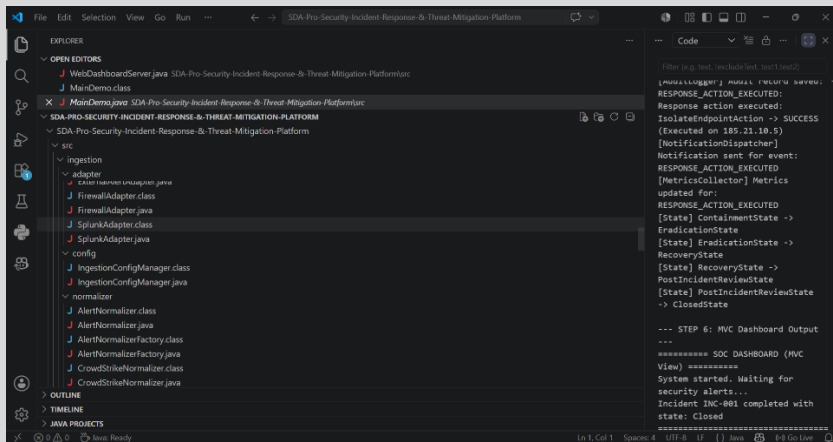
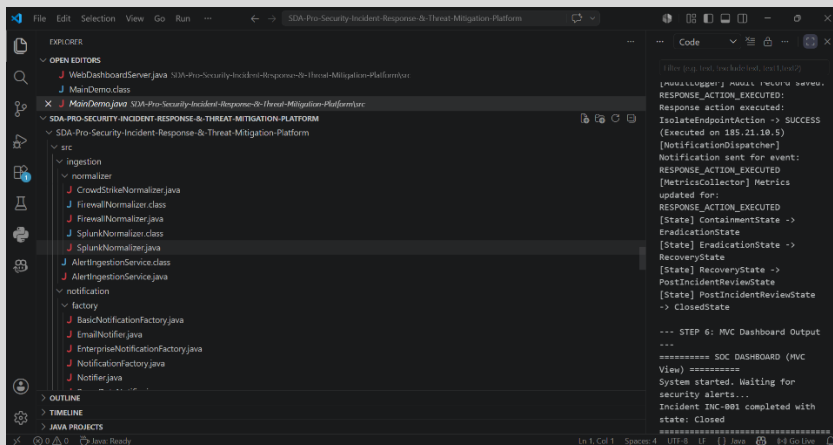
Component	Details
Language	Java (JDK 11 or later)
IDE	Visual Studio Code with Java Extension Pack

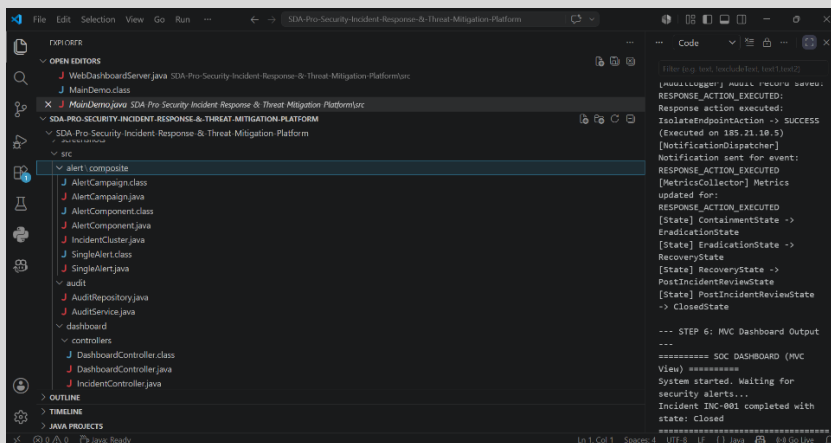
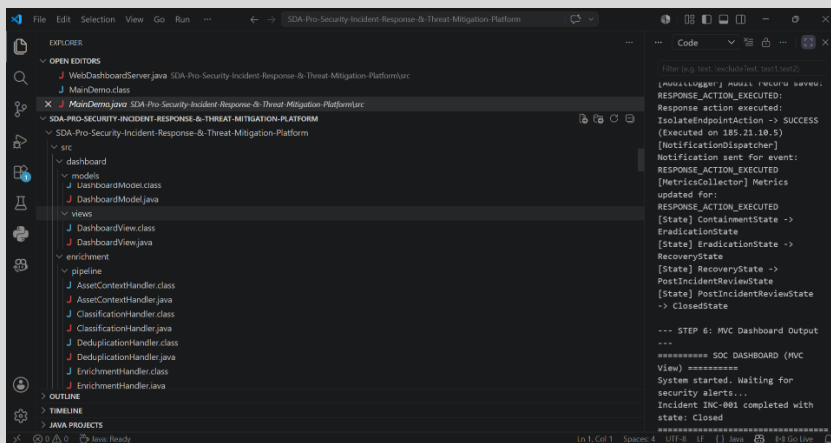
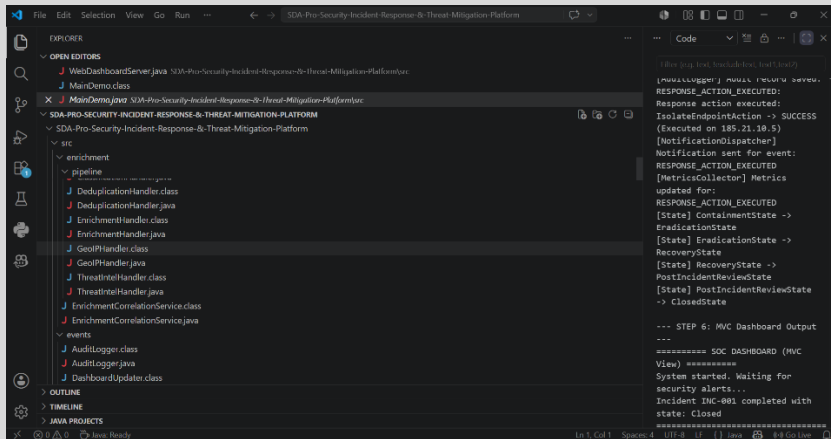
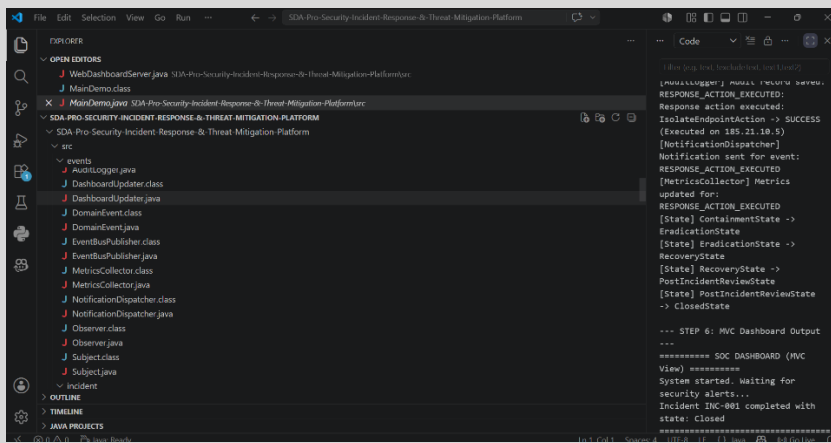
Build Tool	Manual javac compilation (no Maven/Gradle required)
UML Tooling	Enterprise Architect for final diagrams; PlantUML source files in docs/uml
Version Control	Git
Compilation Target	out/ directory
Entry Point (Console)	MainDemo.java
Entry Point (Web GUI)	WebDashboardServer.java (port 9090)

10.2 IDE / VS Code Setup Screenshot









10.3 Compilation Steps

The project is compiled using the following commands from the project root directory:

Linux / macOS / Git Bash

```
# Compile all Java source files
javac -d out $(find src -name "*.java")

# Run the console demo
java -cp out MainDemo

# Run the web dashboard server
java -cp out WebDashboardServer
```

Windows PowerShell

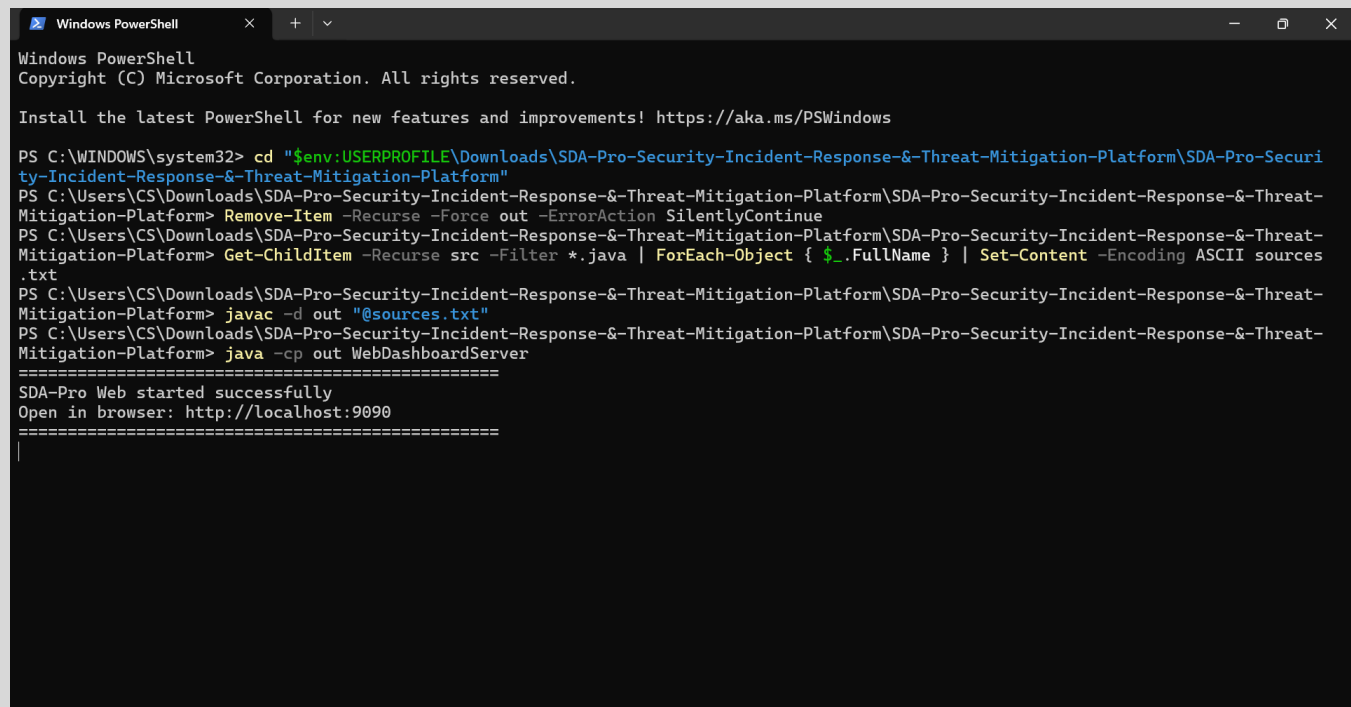
```
# Collect all source file paths
Get-ChildItem -Recurse src -Filter *.java | ForEach-Object { $_.FullName } > sources.txt

# Compile using sources list
javac -d out @sources.txt

# Run the console demo
java -cp out MainDemo

# Run the web dashboard server
java -cp out WebDashboardServer
```

10.4 Java Compilation Screenshot



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> cd "$env:USERPROFILE\Downloads\SDA-Pro-Security-Incident-Response-&-Threat-Mitigation-Platform\SDA-Pro-Security-Incident-Response-&-Threat-Mitigation-Platform"
PS C:\Users\CS\Downloads\SDA-Pro-Security-Incident-Response-&-Threat-Mitigation-Platform\SDA-Pro-Security-Incident-Response-&-Threat-Mitigation-Platform> Remove-Item -Recurse -Force out -ErrorAction SilentlyContinue
PS C:\Users\CS\Downloads\SDA-Pro-Security-Incident-Response-&-Threat-Mitigation-Platform\SDA-Pro-Security-Incident-Response-&-Threat-Mitigation-Platform> Get-ChildItem -Recurse src -Filter *.java | ForEach-Object { $_.FullName } | Set-Content -Encoding ASCII sources.txt
PS C:\Users\CS\Downloads\SDA-Pro-Security-Incident-Response-&-Threat-Mitigation-Platform\SDA-Pro-Security-Incident-Response-&-Threat-Mitigation-Platform> javac -d out "@sources.txt"
PS C:\Users\CS\Downloads\SDA-Pro-Security-Incident-Response-&-Threat-Mitigation-Platform\SDA-Pro-Security-Incident-Response-&-Threat-Mitigation-Platform> java -cp out WebDashboardServer
=====
SDA-Pro Web started successfully
Open in browser: http://localhost:9090
=====
|
```

11. Program Output & Demo Results

The following section documents the expected program output of the SDA-Pro console demonstration, verifying that all five stages of the end-to-end pipeline execute correctly. The demo is driven by MainDemo.java and exercises all twelve design patterns in sequence.

11.1 Program Output Screenshot

```
[Running] cd "c:\Users\CS\Downloads\SDA-Pro-Security-Incident-Response-&-Threat-Mitigation-Platform\src" && java MainDemo.java && java MainDemo

=====
SDA-Pro Security Incident Response & Threat Mitigation Platform
=====

--- STEP 1: Alert Ingestion from 2 Sources ---
[Adapter] Fetching alert from Splunk SIEM mock source...
[Factory Method] SplunkNormalizer converted raw alert to CanonicalAlert.
[Observer] Publishing event -> ALERT_INGESTED: Alert ingested from Splunk SIEM
[DashboardUpdater] Dashboard updated: Alert ingested from Splunk SIEM
[Adapter] Fetching alert from Firewall mock source...
[Factory Method] FirewallNormalizer converted raw alert to CanonicalAlert.
[Observer] Publishing event -> ALERT_INGESTED: Alert ingested from Palo Alto Firewall
[DashboardUpdater] Dashboard updated: Alert ingested from Palo Alto Firewall

--- STEP 2: Composite Grouping ---
[Composite] Added alert to campaign: Possible Credential Attack Campaign
[Composite] Added alert to campaign: Possible Credential Attack Campaign
AlertCampaign: Possible Credential Attack Campaign severity=HIGH
SingleAlert: CanonicalAlert(ALERT-SPL-001, SPLUNK, severity=HIGH, src=185.21.10.5)
SingleAlert: CanonicalAlert(ALERT-FW-001, FIREWALL, severity=MEDIUM, src=45.66.77.88)

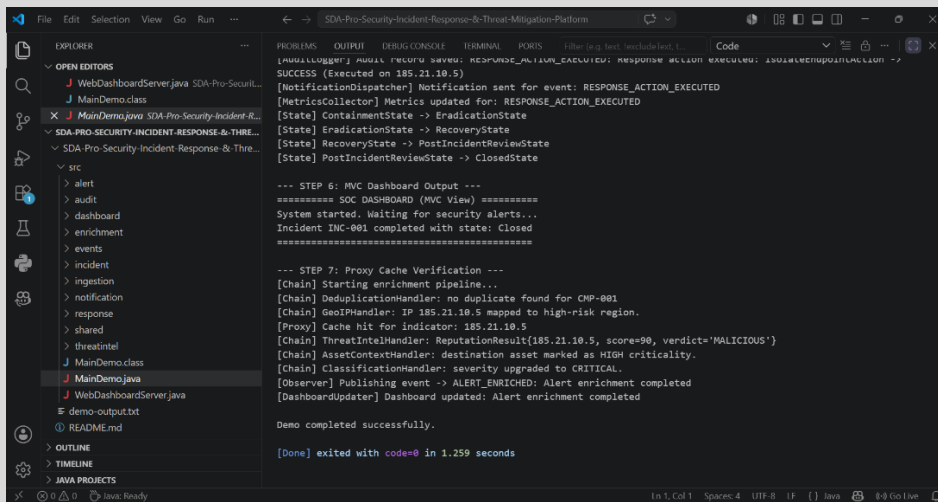
--- STEP 3: Enrichment Pipeline ---
[Chain] Starting enrichment pipeline...
[Chain] DeduplicationHandler: no duplicate found for CMP-001
[Chain] GeoIPHandler: IP 185.21.10.5 mapped to high-risk region.
[Proxy] Cache miss. Calling real threat intel provider...
[Adapter] VirusTotalAdapter queried for: 185.21.10.5
[Chain] ThreatIntelHandler: ReputationResult(185.21.10.5, score=90, verdict='MALICIOUS')
[Chain] AssetContextHandler: destination asset marked as HIGH criticality.
[Chain] ClassificationHandler: severity upgraded to CRITICAL.
[Observer] Publishing event -> ALERT_ENRICHED: Alert enrichment completed
[DashboardUpdater] Dashboard updated: Alert enrichment completed

--- STEP 4: Incident Creation and State Lifecycle ---
[State] Incident created in NewState.
[Observer] Publishing event -> INCIDENT_CREATED: Incident created: INC-001
[DashboardUpdater] Dashboard updated: Incident created: INC-001
[AuditLogger] Audit record saved: INCIDENT_CREATED: Incident created: INC-001
[State] NewState -> UnderTriageState
[State] UnderTriageState -> ContainmentState

--- STEP 5: Response Orchestration ---
[Facade] Starting incident response orchestration for INC-001
[Strategy] Aggressive Containment selected for CRITICAL incident.
[Proxy] Checking authorization for role: SOC_MANAGER
[Decorator] ApprovalGateDecorator: approval granted.
[Decorator] AuditLogDecorator: pre-action audit log.
[Decorator] RollbackDecorator: rollback snapshot saved.
[Action] Blocking malicious IP: 185.21.10.5
[Decorator] AuditLogDecorator: post-action audit log.
[Decorator] MetricsDecorator: action took 1 ms.

[Decorator] MetricsDecorator: action took 1 ms.
[Observer] Publishing event -> RESPONSE_ACTION_EXECUTED: Response action executed: BlockIPAction -> SUCCESS
(Executed on 185.21.10.5)
[DashboardUpdater] Dashboard updated: Response action executed: BlockIPAction -> SUCCESS (Executed on 185.21.10.5)
[AuditLogger] Audit record saved: RESPONSE_ACTION_EXECUTED: Response action executed: BlockIPAction -> SUCCESS
(Executed on 185.21.10.5)
[NotificationDispatcher] Notification sent for event: RESPONSE_ACTION_EXECUTED
[MetricsCollector] Metrics updated for: RESPONSE_ACTION_EXECUTED
[Proxy] Checking authorization for role: SOC_MANAGER
[Decorator] ApprovalGateDecorator: approval granted.
[Decorator] AuditLogDecorator: pre-action audit log.
[Decorator] RollbackDecorator: rollback snapshot saved.
[Action] Isolating endpoint: 185.21.10.5
[Decorator] AuditLogDecorator: post-action audit log.
[Decorator] MetricsDecorator: action took 2 ms.
[Observer] Publishing event -> RESPONSE_ACTION_EXECUTED: Response action executed: IsolateEndpointAction -> SUCCESS
(Executed on 185.21.10.5)
[DashboardUpdater] Dashboard updated: Response action executed: IsolateEndpointAction -> SUCCESS (Executed on 185.21.10.5)
[AuditLogger] Audit record saved: RESPONSE_ACTION_EXECUTED: Response action executed: IsolateEndpointAction -> SUCCESS (Executed on 185.21.10.5)
[NotificationDispatcher] Notification sent for event: RESPONSE_ACTION_EXECUTED
[MetricsCollector] Metrics updated for: RESPONSE_ACTION_EXECUTED
[State] ContainmentState -> EradicationState
[State] EradicationState -> RecoveryState
[State] RecoveryState -> PostIncidentReviewState
[State] PostIncidentReviewState -> ClosedState

--- STEP 6: MVC Dashboard Output ---
===== SOC DASHBOARD (MVC View) =====
```



```
[DEBUG] Audit record saved: RESPONSE_ACTION_EXECUTED: RESPONSE_ACTION_EXECUTED: 13014REINFORCINGACTION -> SUCCESS (Executed on 185.21.10.5)
[NotificationDispatcher] Notification sent for event: RESPONSE_ACTION_EXECUTED
[MetricsCollector] Metrics updated for: RESPONSE_ACTION_EXECUTED
[State] ContainmentState -> EradicationState
[State] EradicationState -> RecoveryState
[State] RecoveryState -> PostIncidentReviewState
[State] PostIncidentReviewState -> ClosedState

--- STEP 8: SOC Dashboard Output ---
===== SOC DASHBOARD (SOC View) =====
System started. Waiting for security alerts...
Incident INC-001 completed with state: Closed
=====

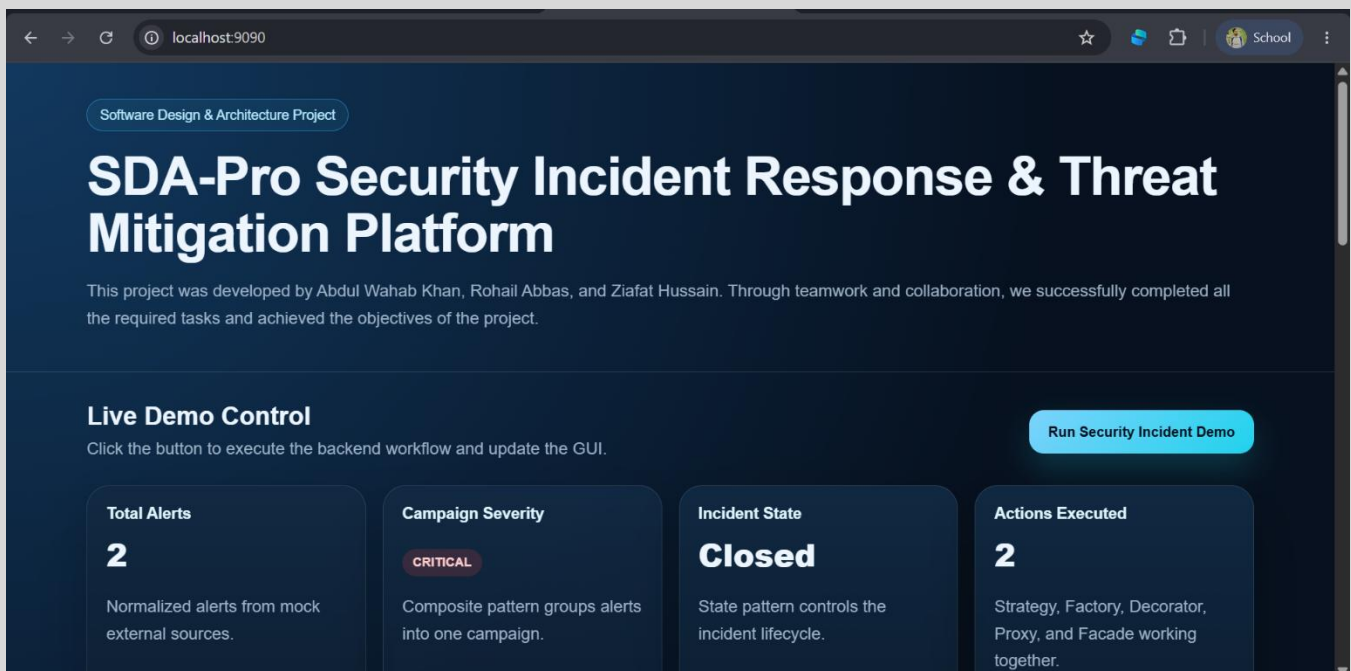
--- STEP 7: Proxy Cache Verification ---
[Chain] Starting enrichment pipeline...
[Chain] DeduplicationHandler: no duplicate found for CMP-001
[Chain] GeoIPHandler: IP 185.21.10.5 mapped to high-risk region.
[Proxy] Cache hit for indicator: 185.21.10.5
[Chain] ThreatIntelHandler: ReputationResult{185.21.10.5, score=90, verdict='MALICIOUS'}
[Chain] AssetContextHandler: destination asset marked as HIGH criticality.
[Chain] ClassificationHandler: severity upgraded to CRITICAL.
[Observer] Publishing event -> ALERT_ENRICHED: Alert enrichment completed
[DashboardUpdater] Dashboard updated: Alert enrichment completed

Demo completed successfully.

[Done] exited with code=0 in 1.259 seconds
```

11.2 Web Dashboard Screenshot

SDA-Pro includes an optional web-based GUI dashboard implemented in `WebDashboardServer.java`. The dashboard renders real-time event updates, alert counts, incident status, and response action outcomes in a browser interface accessible at `http://localhost:9090`.



←→↻🏠📍localhost:9090

☆🔖🔒👤⋮

Architecture Flow

Adapter
External Alerts

Factory Method
Normalize

Composite
Campaign

Chain + Proxy
Enrichment

State
Incident Lifecycle

Facade
Response

Normalized Alerts

ID	Source	Source IP	User	Severity
ALERT-SPL-001	SPLUNK	185.21.10.5	admin	CRITICAL
ALERT-FW-001	FIREWALL	45.66.77.88	unknown	CRITICAL

Incident Summary

Incident ID	INC-001
Title	Suspicious multi-source security incident
Severity	CRITICAL
Final State	Closed
Campaign	CMP-001 — Possible Credential Attack Campaign

←→↻🏠📍localhost:9090

☆🔖🔒👤⋮

Response Actions

Action	Status	Message
BlockIPAction	SUCCESS	Executed on 185.21.10.5
IsolateEndpointAction	SUCCESS	Executed on 185.21.10.5

Backend Console Log

[Adapter] Fetching alert from Splunk SIEM mock source...

[Factory Method] SplunkNormalizer converted raw alert to CanonicalAlert.

[Observer] Publishing event -> ALERT_INGESTED: Alert ingested from Splunk SIEM

[DashboardUpdater] Dashboard updated: Alert ingested from Splunk SIEM

[Adapter] Fetching alert from Firewall mock source...

[Factory Method] FirewallNormalizer converted raw alert to CanonicalAlert.

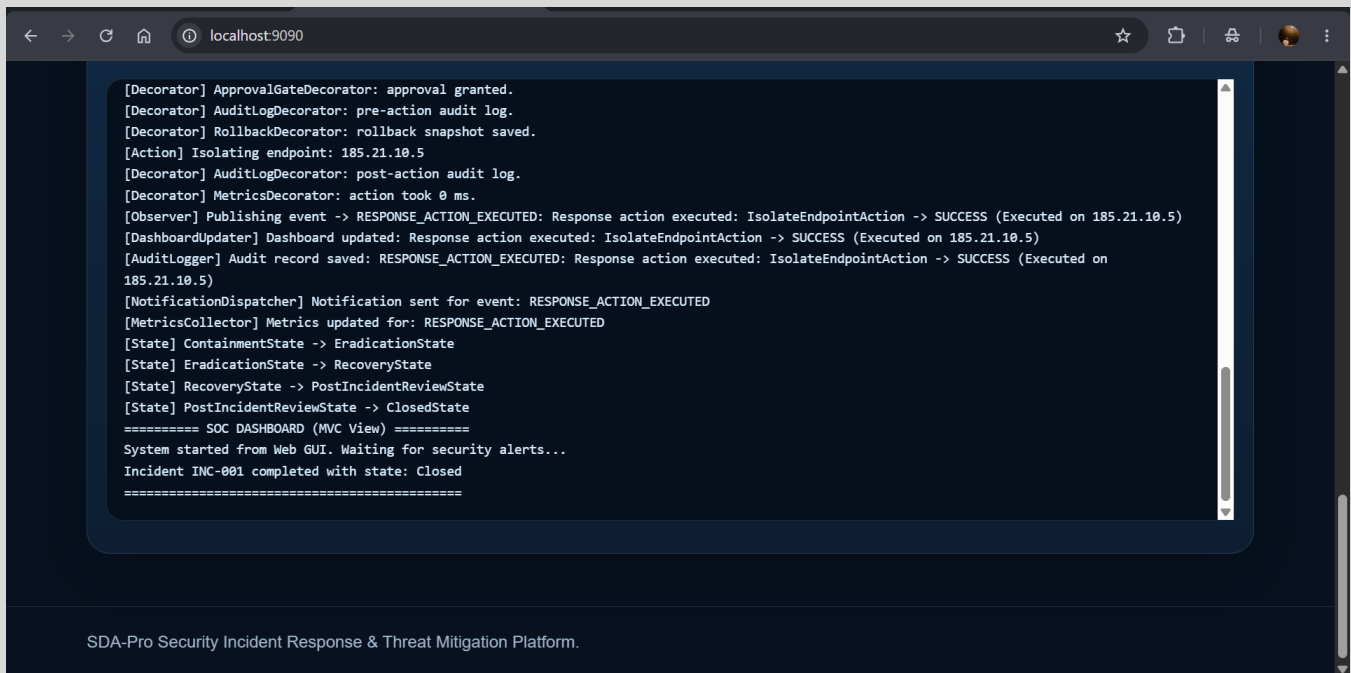
[Observer] Publishing event -> ALERT_INGESTED: Alert ingested from Palo Alto Firewall

[DashboardUpdater] Dashboard updated: Alert ingested from Palo Alto Firewall

[Composite] Added alert to campaign: Possible Credential Attack Campaign

[Composite] Added alert to campaign: Possible Credential Attack Campaign

[Chain] Starting enrichment pipeline...



11.3 Design Pattern Verification Table

The following table maps each design pattern to the specific console output lines that confirm its execution during the demo:

Pattern	Confirming Output Line
Adapter	[Adapter] Fetching alert from Splunk SIEM mock source...
Factory Method	[Factory Method] SplunkNormalizer converted raw alert to CanonicalAlert.
Composite	[Composite] Added alert to campaign: Possible Credential Attack Campaign
Chain of Responsibility	[Chain] DeduplicationHandler / GeolPHandler / ThreatIntelHandler ...
Proxy (Caching)	[Proxy] Cache miss. Calling real threat intel provider...
Proxy (Authorization)	[Proxy] Checking authorization for role: SOC_MANAGER
State	[State] NewState -> UnderTriageState -> ContainmentState
Facade	[Facade] Starting incident response orchestration for INC-001
Strategy	[Strategy] Aggressive Containment selected for CRITICAL incident.
Decorator (Approval)	[Decorator] ApprovalGateDecorator: approval granted.
Decorator (Audit)	[Decorator] AuditLogDecorator: pre-action audit log.
Decorator (Rollback)	[Decorator] RollbackDecorator: rollback snapshot saved.
Decorator (Metrics)	[Decorator] MetricsDecorator: action took 2 ms.
Observer	[Observer] Publishing event -> RESPONSE_ACTION_EXECUTED ...
Singleton	EventBusPublisher / ThreatIntelCache — shared single instance used throughout

12. Testing & Validation

SDA-Pro validation is performed through the MainDemo.java integration demonstration, which exercises the full pipeline end-to-end and verifies correctness through console output inspection. The prototype does not include a formal unit test suite, but the demo is structured to provide clear per-step verification.

13. Challenges & Lessons Learned

13.1 Challenges Encountered

- **Pattern Identification:** Determining precisely which design patterns to apply to each subsystem required multiple iterations of design review. The initial design used conditional if-else chains in several places where patterns such as Factory Method and Strategy were ultimately more appropriate.
- **Decorator Ordering:** Establishing the correct stacking order for response action decorators (Rollback → AuditLog → ApprovalGate → Metrics) required careful consideration of which concerns should execute first and whether failed inner actions should still generate audit records.
- **State Machine Completeness:** Ensuring that all six IncidentState implementations correctly rejected invalid transitions without causing NullPointerExceptions required disciplined implementation of all transition methods in every state class.
- **Event Bus Decoupling:** Early versions of the system had direct method calls from IncidentManagementService to AuditService and DashboardController. Refactoring to use EventBusPublisher required rethinking the data flow and ensuring event payloads carried sufficient information for all subscribers to act without calling back into the publisher.
- **In-Memory Cache Thread Safety:** ThreatIntelCache used a plain HashMap in early versions. Under concurrent simulation, this caused occasional stale reads. The fix was to use ConcurrentHashMap.

13.2 Lessons Learned

- **Architecture First:** Starting with a clear layered architecture diagram before writing code prevented several circular dependency issues that arose in early prototypes.
- **Pattern Documentation:** Writing brief comments in each class explaining which pattern it implements and why made code review and viva preparation significantly easier.
- **ADR Value:** Drafting Architecture Decision Records forced the team to articulate the rationale for each design choice. This discipline revealed two cases where the original choice was suboptimal and led to better decisions.
- **Facade as Glue:** The IncidentResponseFacade proved to be one of the most valuable additions to the design. It reduced the complexity of the calling code dramatically and made it easy to understand the response flow at a high level without needing to trace through strategy, factory, decorator, and proxy classes individually.
- **Observer Scalability:** The Observer pattern made it trivial to add the MetricsCollector subscriber late in development. No existing code required modification, demonstrating the practical value of loose coupling.

14. Conclusion

SDA-Pro successfully delivers a clean, well-structured, and fully demonstrable Java prototype of a Security Incident Response and Threat Mitigation Platform. The system achieves all defined project objectives and satisfies the complete set of design and architecture requirements specified in class.

Twelve design patterns are applied purposefully throughout the codebase: three creational patterns govern object creation in the ingestion, notification, and threat intelligence subsystems; five structural patterns organize the alert composite hierarchy, external system integration, caching, authorization, and response action enhancement; and four behavioral patterns control alert enrichment, incident lifecycle, response strategy selection, and real-time event notification.

Four architectural styles are demonstrated at different levels of the system. Service-Oriented Architecture defines the macro structure of the platform. Layered Architecture enforces dependency discipline within the codebase. Model-View-Controller provides a clean separation in the dashboard subsystem. Event-Driven Architecture decouples all side-effect notifications from the core business logic.

Five Architecture Decision Records document the reasoning behind the most significant design choices, providing a traceable record of how the architecture evolved and why certain alternatives were rejected. Six API contracts define the service interfaces in sufficient detail to support future integration or conversion to a genuine microservices deployment.

The prototype is deliberately straightforward in implementation while being rigorous in design. It is designed to be explainable at every level, from the high-level SOA component diagram down to the individual method calls visible in the console output. This combination of architectural clarity and implementation simplicity makes SDA-Pro a solid demonstration of software design and architecture principles.

END OF REPORT

SDA-Pro — Security Incident Response & Threat Mitigation Platform
Abdul Wahab Khan (55072) | Rohail Abbas (54729) | Ziafat Hussain (55062)
Riphah International University, Islamabad | Spring 2026